Sistema de detección temprana del riesgo académico en estudiantes universitarios mediante Machine Learning

Integrantes:
Valentina Bello Olarte
William Gómez Marín

Programa:
Ciencia de Datos
Universidad Externado de Colombia

Asignatura:
Machine Learning II

Docente:
Wilmer Dario Pineda Rios

Entrega:
Segunda entrega del Proyecto
Datos, modelamiento y validación

Bogotá, Colombia
2026

1. Consolidación de datos

La función principal de nuestra base de datos es que funcione a la hora de desarrollar un modelo de Machine Learning orientado a la predicción del riesgo de deserción universitaria, buscando identificar estudiantes con mayor riesgo a partir de sus características académicas, socioeconómicas y de experiencia universitaria.

Para el proyecto se seleccionó el conjunto **"*Risk of Desertion in Undergraduate UCC*"**, el cual se consiguió en Mendeley Data, gracias a su relación directa con la deserción universitaria y su contexto colombiano en la Universidad Cooperativa de Colombia (UCC), la base original está distribuida en ***X_train_balanced.xlsx, y_train_balanced.xlsx, X_test.xlsx y y_test.xlsx,*** *me*diante Python se integraron las variables predictoras con su variable objetivo y se consolidaron los conjuntos de entrenamiento y prueba, obteniendo 4.000 registros reales y 331 variables predictoras.

La variable objetivo, ***nivel_motivacion***, ya estaba en la base original y representa el nivel de riesgo de deserción de cada estudiante. Contiene cinco categorías:

Categoría original	En riesgo / No en riesgo
Deserción riesgo muy bajo	**No en riesgo**
Deserción riesgo leve	**No en riesgo**
Deserción riesgo medio	**En riesgo**
Deserción riesgo alto	**En riesgo**
Deserción casi segura	**En riesgo**

Para cumplir con el mínimo de 5.000 registros, se generaron mediante Python 1.000 registros sintéticos a partir de combinaciones de perfiles reales pertenecientes a una misma categoría de ***nivel_motivacion***. Para cada nuevo registro sintético se seleccionaron dos perfiles reales de la misma categoría de riesgo y, luego, las variables predictoras se organizaron en grupos según su estructura de nombres, así en cada grupo se tomaron los valores de uno de los dos perfiles seleccionados, combinando así características de ambos estudiantes para construir un nuevo registro, este procedimiento permitió generar nuevas combinaciones de características sin copiar completamente un registro real, también en cada registro sintético fue comparado con los registros reales y con los demás sintéticos para evitar copias exactas y duplicados, luego al hacer la auditoría también se identificó una columna duplicada y variables potencialmente relacionadas con data leakage, principalmente asociadas con cancelación, abandono o cambio de carrera, estas fueron excluidas, acá se obtuvo una base final de 5.000 registros, 319 variables predictoras, 4.000 registros reales y 1.000 sintéticos, sin datos faltantes.

La validación mostró una diferencia promedio de 0,0054 en las proporciones de las variables, lo que indica que la distribución de las características en los datos sintéticos es muy similar a la de los datos reales, la diferencia media de correlaciones fue de 0,0308, esto nos muestra que las relaciones entre las variables se conservaron de forma cercana, y el 94,94 % de los pares presentó una diferencia de correlación <= 0,10, lo que confirma la similitud, la diferencia promedio de la relación de las variables con nivel_motivacion fue de 0,0245, mostrando que la relación de las características con la variable objetivo también se mantuvo relativamente cercana, para concluir no se encontraron copias exactas entre registros reales y sintéticos ni duplicados entre los sintéticos, evitando que la información sintética fuera simplemente una repetición de los datos originales.

2. Flujo reproducible sin fuga de datos

Para evitar la fuga de información (data leakage), se usó un flujo reproducible mediante Pipeline. Primeramente, se separaron los datos en 80 % para entrenamiento (4.000 registros) y 20 % para prueba (1.000 registros), manteniendo la misma proporción entre estudiantes en riesgo y no en riesgo mediante stratify, está funcion nos ayuda a dividir los datos y conservar aproximadamente la misma proporción de estudiantes en riesgo y no en riesgo tanto en entrenamiento como en prueba.

Debido a que las variables ya se encontraban codificadas y no existían datos faltantes, no fue necesario aplicar nuevas etapas de codificación o imputación, para el modelo SVM se usó el escalamiento dentro del Pipeline, para que sus parámetros se aprendieran únicamente a partir de los datos de entrenamiento, luego, el modelo fue entrenado con estos datos y evaluado sobre el conjunto de prueba, que permaneció aislado durante el aprendizaje.

El flujo utilizado fue:

Separación de datos - > escalamiento - > entrenamiento del modelo - > predicción - > evaluación.

Como prueba inicial, el SVM obtuvo un 97 % de acurracy sobre el conjunto de prueba y un recall de 96 % para los estudiantes en riesgo, identificando correctamente 570 de los 591 casos reales de esta clase, estos resultados corresponden a una primera prueba del flujo y luego serán "comparados" a través de validación y otros modelos.

3. Reproducción y adaptación del método

El artículo que tenemos como guía plantea la detección temprana de estudiantes at-risk mediante clasificación supervisada y compara SVM, Naive Bayes, KNN, regresión logística, árbol de decisión y Random Forest. Además aplica selección de variables, técnicas para el desbalance y validación cruzada, nosotros conservamos el propósito de identificar estudiantes en riesgo, pero lo adaptamos a la base colombiana de la Universidad Cooperativa de Colombia (UCC).

Al igual que en el artículo nosotros usamos una variable binaria a partir de nivel_motivacion donde riesgo medio, riesgo alto y casi segura se consideran en riesgo y se les pone 1, mientras que riesgo muy bajo y riesgo leve se consideran no riesgo y se les pone 0. Esto no es lo mismo que está en el artículo, porque la base UCC que es nuestra base no contiene la definición por curso utilizada allí, no contiene notas. Se compararon un clasificador de mayoría como el baseline o línea base, regresión logística, SVM con kernel RBF, Random Forest, HistGradientBoosting y Random Forest con selección de 30 variables. La selección de las 30 variables se incorporó dentro del Pipeline como una adaptación de la estrategia de reducción de dimensionalidad del artículo de referencia, permitiendo que el modelo se concentre en la información más importante. Por otro lado, el algoritmo HistGradientBoosting se integró como una técnica de aprendizaje que no está en el paper guía para evaluar su capacidad predictiva frente a los demás modelos supervisados.

El ajuste de hiperparámetros se realizó mediante validación cruzada anidada, con cinco pliegues externos y tres pliegues internos. El criterio utilizado para seleccionar configuraciones fue F2, debido a que otorga mayor peso al recall pero igualmente también le ponemos cuidado a la precisión.

Modelo	Configuración seleccionada
SVM	C=10 y gamma= scale más específicamente 0.03
Random forest	200 árboles con max depth none y min samples 2

Boosting	Learning rate 0.1 con max depth 6 y max iter de 200
Random Forest con k=30	200 árboles con max depth none

Los valores están sustentados en el colab.

4. Validación, métricas e interpretación preliminar

En la evaluación usamos validación cruzada anidada sobre el conjunto de entrenamiento y una evaluación única sobre un hold-out agrupado que consta del 20% de los datos. Se reportan accuracy, precision, recall, especificidad, F1, F2 y ROC-AUC. El recall de la clase en riesgo es la métrica que nos interesa porque un falso negativo es un estudiante en riesgo que no sería identificado por el sistema y la idea es no perder ningún estudiante que esté en riesgo, identificarlo a tiempo. F2 se utilizó para seleccionar hiperparámetros, mientras que las demás métricas nos permiten ver el equilibrio entre detección y falsas alarmas.

La línea base de mayoría presenta recall = 1,000 para la clase en riesgo porque predice esa clase para todos los registros ya que tiene en su mayoría el dataset tiene datos en riesgo. Sin embargo, su especificidad es 0, por eso el recall no se interpreta de solo sino que se complementa con precision, especificidad, F1, F2 y ROC-AUC

4.1 Validación cruzada anidada

Modelo	Accuracy	Precisión	Recall	Especificidad	F1	F2	ROC-AUC
Baseline	0,566 ± 0,107	0,566 ± 0,107	1,000 ± 0,000	0,000 ± 0,000	0,718 ± 0,086	0,862 ± 0,050	0,500 ± 0,000
Regresión logística	0,909 ± 0,038	0,917 ± 0,052	0,909 ± 0,056	0,906 ± 0,026	0,913 ± 0,049	0,910 ± 0,052	0,964 ± 0,010
SVM	0,932 ± 0,026	0,938 ± 0,027	0,933 ± 0,043	0,925 ± 0,011	0,935 ± 0,033	0,934 ± 0,039	0,981 ± 0,008
Random forest	0,949 ± 0,018	0,982 ± 0,018	0,920 ± 0,030	0,983 ± 0,016	0,950 ± 0,024	0,932 ± 0,028	0,991 ± 0,005
Boosting	0,950 ± 0,022	0,950 ± 0,028	0,958 ± 0,024	0,941 ± 0,031	0,954 ± 0,022	0,956 ± 0,022	0,987 ± 0,007
Random forest con k=30	0,876 ± 0,023	0,905 ± 0,034	0,870 ± 0,051	0,886 ± 0,023	0,886 ± 0,026	0,876 ± 0,040	0,949 ± 0,017

En la validación anidada, HistGradientBoosting obtuvo F2 = 0,956 ± 0,022 y recall = 0,958 ± 0,024. Random Forest obtuvo ROC-AUC = 0,991 ± 0,005 y especificidad = 0,983 ± 0,016. El modelo RF con selección top-30 presentó F2 = 0,876 ± 0,040, mostrando que la reducción a 30 variables produjo una disminución del desempeño frente a los modelos que utilizan el conjunto completo de predictores.

4.2 Evaluación en hold-out

Modelo	Accuracy	Precisión	Recall	Especificidad	F1	F2	ROC-AUC
Baseline	0,607	0,607	1,000	0,000	0,756	0,885	0,500
Regresión logística	0,938	0,929	0,972	0,885	0,950	0,963	0,962
SVM	0,961	0,971	0,964	0,956	0,968	0,966	0,993

Random forest	0,958	0,994	0,937	0,991	0,965	0,948	0,996
Boosting	0,967	0,969	0,976	0,952	0,973	0,975	0,994
Random forest con k=30	0,932	0,940	0,949	0,906	0,944	0,947	0,965

El modelo seleccionado es Boosting (HistGB), debido a su F2 medio en la validación anidada. En el hold-out agrupado de 1.105 registros obtuvo accuracy = 0,967, precision = 0,969, recall = 0,976, especificidad = 0,952, F1 = 0,973, F2 = 0,975 y ROC-AUC = 0,994. Este conjunto permaneció separado del proceso de ajuste de hiperparámetros.

4.3 Errores y estabilidad

La matriz de confusión del modelo HistGradientBoosting en el hold-out fue: 413 verdaderos negativos, 21 falsos positivos, 16 falsos negativos y 655 verdaderos positivos. De los 671 estudiantes realmente clasificados como en riesgo, el modelo identificó 655 (97,6 %) y no identificó 16 (2,4 %). Entre los 434 estudiantes sin riesgo, 21 fueron señalados como en riesgo (4,8 %).

Categoría original	n	Errores	Tasa de error
Deserción riesgo muy bajo	196	2	1,0 %
Deserción riesgo leve	238	19	8,0 %
Deserción riesgo medio	204	13	6,4 %
Deserción riesgo alto	273	2	0,7 %
Deserción casi segura	194	1	0,5 %

La mayor tasa de error se observó en la categoría de riesgo leve seguido de en riesgo medio. Debemos recordar que ambas categorías se encuentran alrededor del límite utilizado para transformar las cinco categorías originales en una clasificación binaria.

4.4 Comportamiento en registros reales y en registros sintéticos

Fuente	n	Accuracy	Recall	Precision	F1
REAL	890	0,975	0,989	0,971	0,980
SINTÉTICO	215	0,930	0,919	0,958	0,938

El desempeño fue diferente entre los registros reales y sintéticos. El modelo presentó mejores resultados en los registros reales mientras que en los sintéticos el desempeño fue menor. Por eso se debe tratar los registros generados como una ampliación aproximada y no como si fueran estudiantes reales.

4.5 Interpretación

Para saber cuáles eran las variables que de verdad le importaban al modelo para adivinar el riesgo de los estudiantes, utilizamos dos métodos diferentes que se complementan. El primero consistió en alterar a propósito los datos de cada columna en el grupo de prueba que era el hol-out y mirar qué tanto se desplomaba el puntaje F2 del modelo. Las variables cuya alteración causó las caídas más drásticas que vienen siendo también las más importantes fueron r_carrera, d_acad, per_profesores, autopercepcion y recomendación_u, acompañadas por otras como el apoyo familiar y las horas semanales.

Como segundo método de apoyo usamos SHAP, que analiza cómo actúa el modelo por dentro frente a cada dato. Al cruzar los resultados de ambos procedimientos, salió que ocho variables clave coincidieron en ser de las más importantes, destacando nuevamente el rendimiento de la carrera, el apoyo académico, el entorno del campus y la percepción de los profesores. Esto nos permite concluir que el algoritmo apoya sus decisiones en los aspectos académicos y de entorno más lógicos del estudiante.

4.6 Limitaciones y cierre

La principal limitación es que tuvimos que meter datos sintéticos, que tenían similitud con los datos reales porque fueron evaluados pero que no significa ni cambia que no es la incorporación de estudiantes adicionales observados. También existe una alta repetición de perfiles reales, razón por la cual se utilizó validación agrupada, es decir hay muchos datos de una sola persona que cursa por ejemplo diferentes materias, la validación agrupada controla las copias exactas de perfiles pero los registros sintéticos fueron construidos a partir de perfiles reales y no se conservaron identificadores de parentesco entre los perfiles padres y los registros generados, por eso no se afirma que toda forma posible de dependencia haya sido eliminada.

Con esta fase se completa el modelamiento y la validación exigidos, comparamos modelos frente a una línea base, ajustamos hiperparámetros con validación cruzada anidada, controlamos la repetición de perfiles gracias al agrupamiento, evaluamos el modelo en un hold-out separado y realizamos una interpretación preliminar de las variables relevantes. El despliegue funcional de cada dato y gráfica que se usó acá está disponible en el colab.

5. Referencias

Mondragon Ramirez, H. S., Bohorquez Bernal, N. S., Nova Martínez, M., & Torres-Londoño, C. I. (2025). *Risk of Desertion in Undergraduate UCC* [Data set]. Mendeley Data, Version 1. https://doi.org/10.17632/cz6sxh48jm.1

6. Uso de IA

Durante el desarrollo del proyecto se utilizó IA como herramienta de apoyo en algunas etapas del trabajo, se utilizó para la elaboración y revisión de códigos en Python, incluyendo la consolidación, limpieza y validación de los datos, así como la implementación del flujo reproducible mediante Pipeline.

También se utilizó como apoyo para el análisis e interpretación de algunos resultados obtenidos, con el propósito de comprender mejor las métricas, las validaciones realizadas y su relación con los objetivos del proyecto.

Sin embargo, todo el contenido dado por la IA fue revisado por el grupo conformado por Valentina Bello Olarte y William Gómez Marín